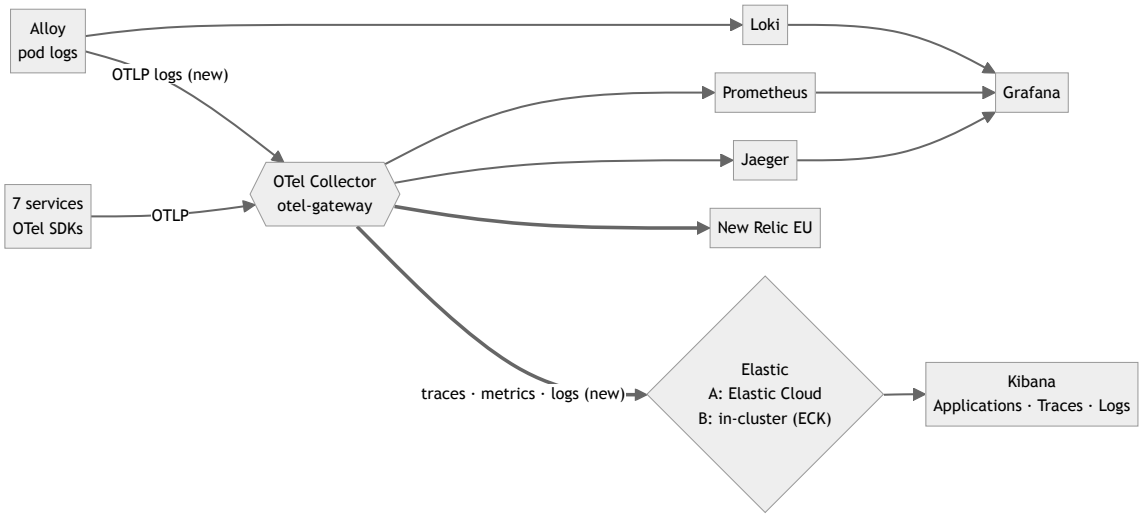
ECOMMERCE-DEMO-K8S · INTEGRATION PLAN

Adding Elastic Observability to the Kubernetes shop

Elastic accepts OpenTelemetry directly, so in both options Elastic becomes one more destination on the existing OTel Collector, next to Jaeger, Prometheus and New Relic. Applications, images, Grafana and New Relic stay unchanged, and everything sits behind an `elastic.enabled` switch.

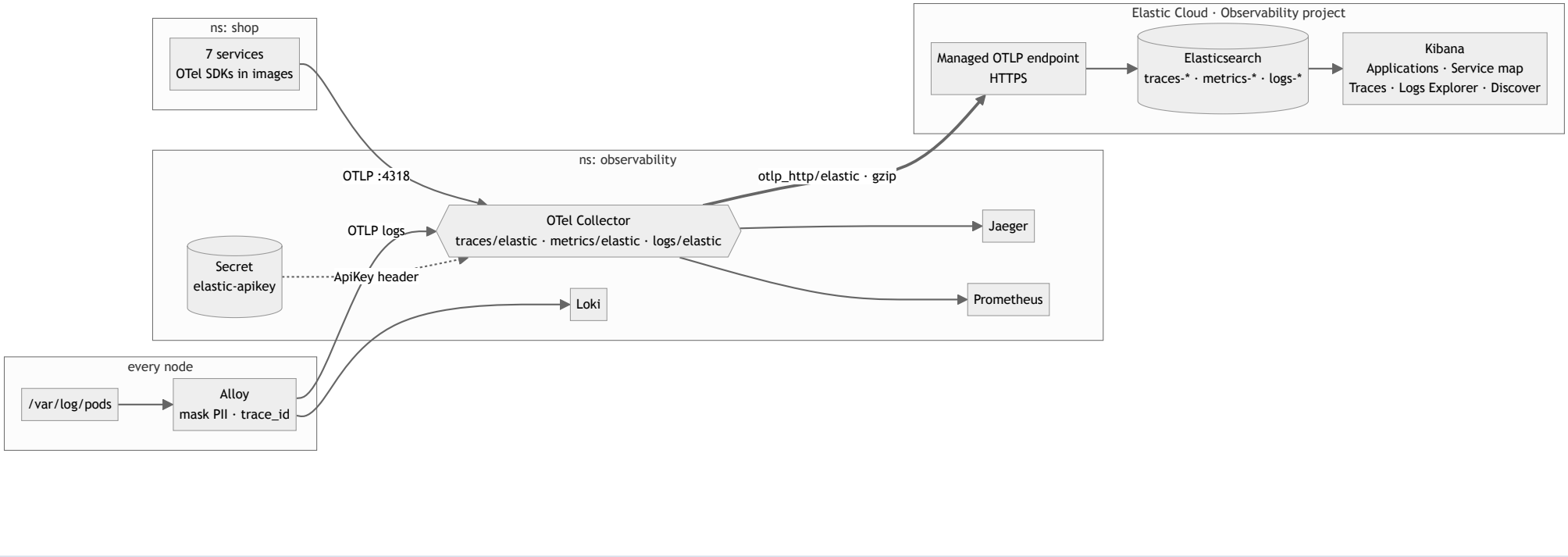


	A · ELASTIC CLOUD (HOSTED)	B · SELF-HOSTED IN KIND (ECK OPERATOR)
Where Elasticsearch + Kibana run	Elastic's cloud (SaaS)	Pods in a new `elastic` namespace, managed by the ECK operator
How data gets there	Collector exporter `otlp_http/elastic` → managed OTLP endpoint, `Authorization: ApiKey`	Collector exporter `elasticsearch` (OTel-native mapping) → `https://elasticsearch-es-http.elastic:9200`, ECK's CA trusted
Effort	Small — same pattern as New Relic	Larger — operator, Elasticsearch, Kibana, TLS, route
Memory on the Mac	None	~3 GB more (Elasticsearch ~2 GB, Kibana ~1 GB)
Cost	14-day free trial, then paid	Free (basic licence includes the APM UI)
Data location	Leaves the cluster	Stays on the laptop
What you learn	Sending to a SaaS backend	A third operator (ECK), Elasticsearch on Kubernetes, in-cluster TLS

Thick arrows: data leaving the collector to a commercial or external backend. Logs to Elastic are new: Alloy gains an OTLP output so logs (already PII-masked

Option A Elastic Cloud (hosted)

The collector sends traces, metrics and logs over HTTPS to the Observability project's managed OTLP endpoint. The API key lives in a Kubernetes Secret, exactly like the New Relic license key.



1 **You** create an Elastic Cloud Observability project and copy its managed OTLP endpoint and an API key (account and login are yours).

2 Store the API key as Secret `elastic-apikey` in `observability`.

3 `charts/observability-config`: `elastic.enabled` switch, exporter `otlp_http/elastic`, pipelines `traces/elastic` and `metrics/elastic`.

4 Alloy: add an OTLP output to the collector; new `logs/elastic` pipeline, so Kibana links logs to traces.

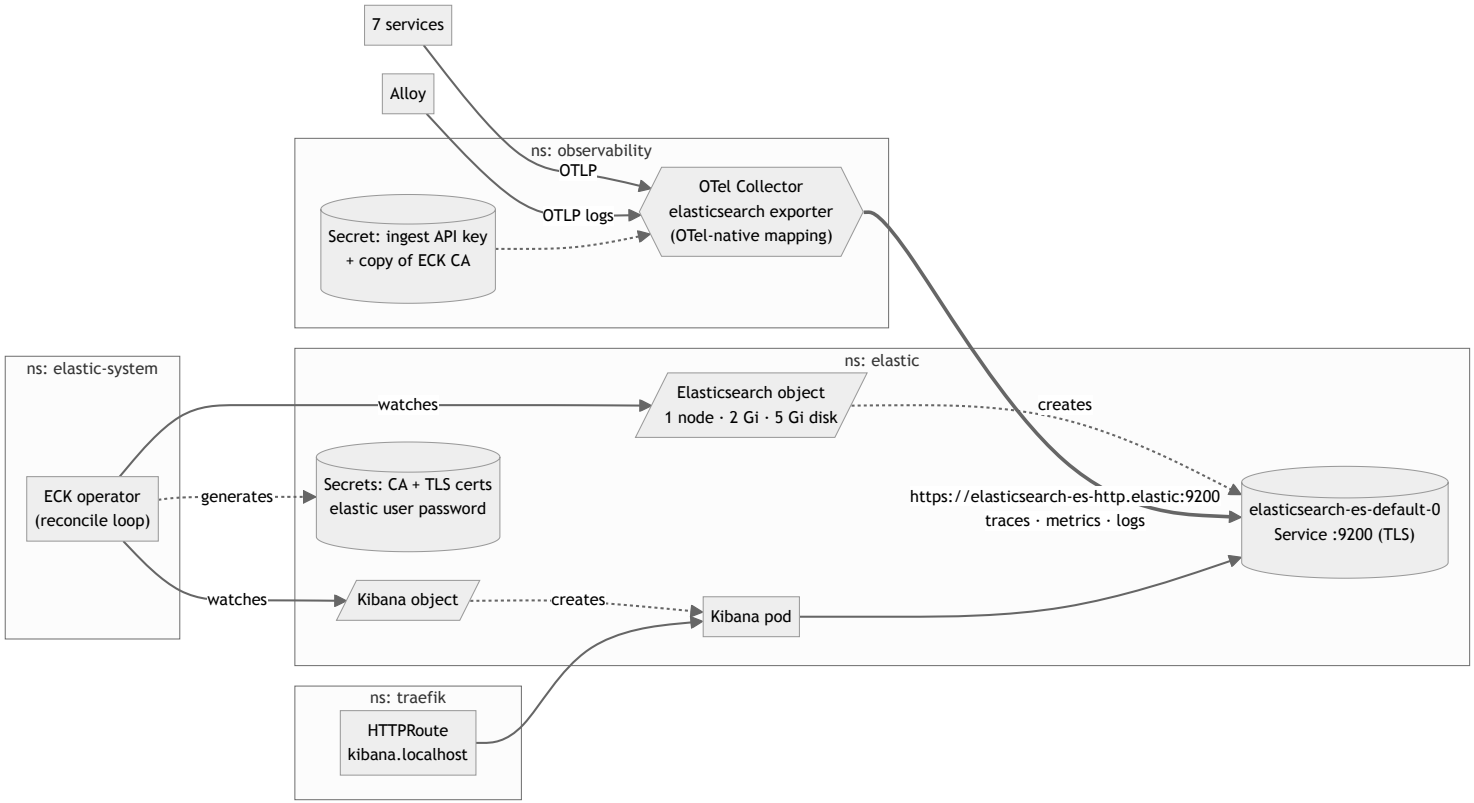
5 `helm upgrade` obs-config and alloy; collector counters: sent > 0, failed = 0.

6 Kibana → Observability → Applications (7 services), Service map, Traces, Logs Explorer, Discover.

7 Document in `docs/OBSERVABILITY.md` and push.

Option B Self-hosted in kind with the ECK operator

The ECK operator turns two small objects (`Elasticsearch` , `Kibana`) into running pods, certificates and passwords. The collector writes straight into Elasticsearch over TLS, trusting the CA that ECK generated.



0 Memory: raise Docker to ~14 GB, or pause something (e.g. New Relic export) while testing.

1 Install the ECK operator (Helm chart `elastic/eck-operator`) in `elastic-system` .

2 New chart `charts/elastic` : `Elasticsearch` (single node) + `Kibana` objects; ECK creates pods, certs and the password.

3 HTTPRoute `kibana.localhost` .

4 Create an ingest API key; store it (and ECK's CA) as Secrets in `observability` .

5 Collector: `elasticsearch` exporter, TLS with ECK's CA (no skip-verify), traces/metrics/logs pipelines behind `elastic.enabled` .

6 Alloy → collector → Elasticsearch for logs.

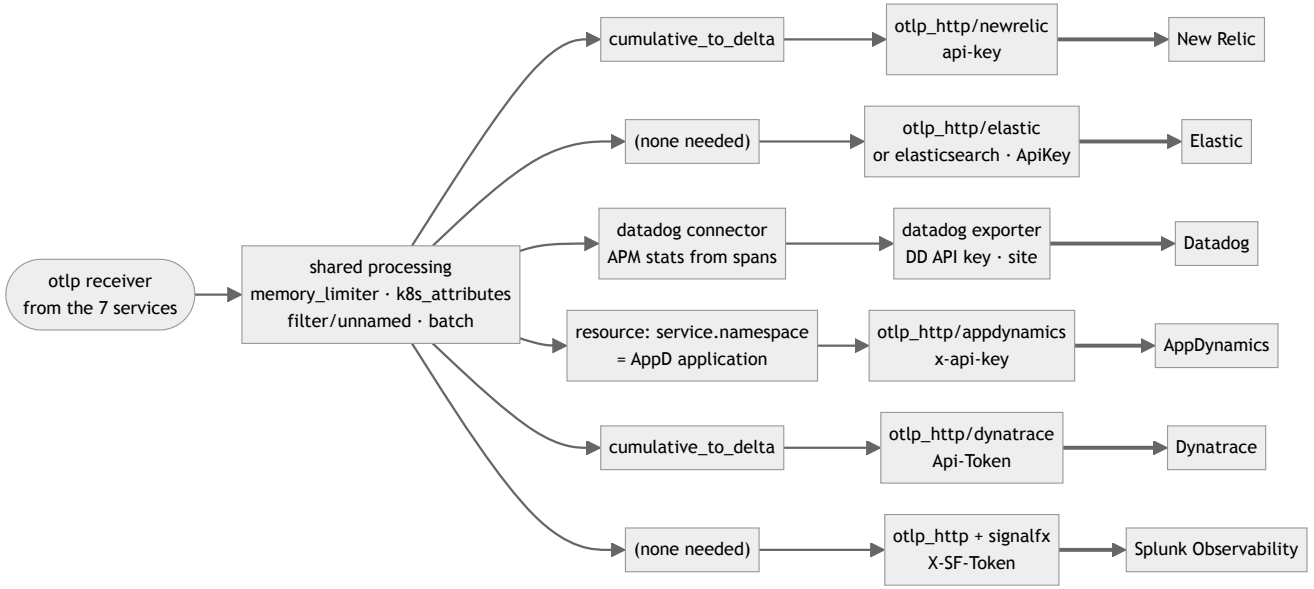
7 Kibana: Applications, Traces, Service map, Logs Explorer, Discover.

8 Optional: Kubernetes metrics (EDOT collector per node: kubelet + cluster receivers).

9 Document and push. Pin Elastic 9.x (≥ 2 weeks old) for native OTel APM views.

Any backend How the collector sends to a vendor: the pattern

Every commercial backend is added the same way: a new pipeline that reuses the shared processing, plus a vendor-specific exporter. Only five things change per vendor; the apps, SDKs, receiver and shared processing never do.



The five things that change per vendor

- **Endpoint** — the vendor's ingest URL, usually region-specific (EU vs US).
- **Auth header** — key or token, always from a Kubernetes Secret via `${env:…}`.
- **Metric temporality** — delta (New Relic, Dynatrace) or cumulative (Prometheus-style backends).
- **Required attributes** — e.g. AppDynamics maps `service.namespace` to its application.
- **Vendor components** — e.g. Datadog's exporter and connector instead of plain OTLP.

A new vendor in `otel-gateway.yaml` (template)

```
exporters:
  otlp_http/vendor:
    endpoint: {{ .Values.vendor.endpoint }}
    headers:
      <auth-header>: ${env:VENDOR_KEY}   # from a Secret
    compression: gzip
service:
  pipelines:
    traces/vendor:
      receivers:  [otlp]
      processors: [memory_limiter, k8s_attributes, batch]
      exporters:  [otlp_http/vendor]
    metrics/vendor:   # add cumulative_to_delta if delta needed
      receivers:  [otlp]
      processors: [memory_limiter, k8s_attributes, batch]
      exporters:  [otlp_http/vendor]
```

Cheat sheet Collector settings per backend

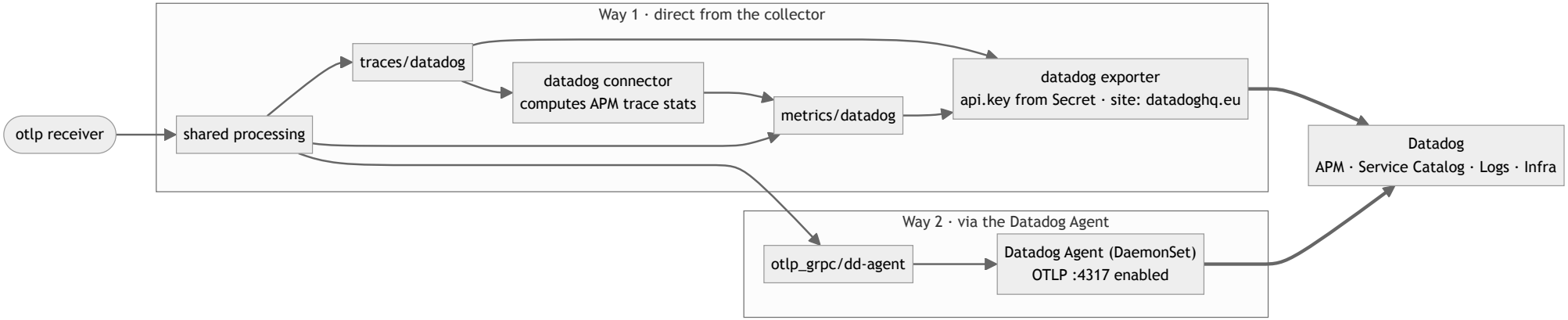
Typical settings for the main backends. Endpoint formats and header names are what each vendor documents for OpenTelemetry ingest; always confirm the exact URL for your account and region in the vendor's console or current docs before using them.

BACKEND	COLLECTOR EXPORTER	ENDPOINT (FORMAT)	AUTH	METRICS	LOGS	NOTES
New Relic	`otlp_http`	`https://otlp.eu01.nr-data.net` (EU) · `otlp.nr-data.net` (US)	`api-key: <license key>`	delta → `cumulative_to_delta`	OTLP	Already running in this cluster
Elastic	`otlp_http` (Cloud) or `elasticsearch` (self-hosted)	project's managed OTLP endpoint · `https://<es>:9200`	`Authorization: ApiKey <key>`	cumulative OK	OTLP / ES	Kibana 9.x shows OTel APM natively
Datadog	`datadog` exporter + `datadog` connector	`site: datadoghq.eu` / `datadoghq.com`	`api.key: <DD API key>`	handled by the exporter	datadog exporter	Connector computes APM trace stats; alternative: send OTLP to the Datadog Agent
AppDynamics (Cisco)	`otlp_http`	OTel endpoint shown in the Controller (region-specific)	`x-api-key: <OTel API key>`	traces are the focus	—	`service.namespace` = application, `service.name` = tier
Dynatrace	`otlp_http`	`https://<env-id>.live.dynatrace.com/api/v2/otlp`	`Authorization: Api-Token <token>`	delta → `cumulative_to_delta`	OTLP	Token needs trace / metric / log ingest scopes
Splunk Observability	`otlp_http` (traces) + `signalfx` (metrics)	`https://ingest.<realm>.signalfx.com/…`	`X-SF-Token: <token>`	signalfx exporter	`splunk_hec`	Splunk ships its own collector distribution
Grafana Cloud	`otlp_http`	`https://otlp-gateway-<zone>.grafana.net/otlp`	basic auth: instance ID + token	cumulative OK	OTLP	Lands in hosted Tempo / Mimir / Loki
Honeycomb	`otlp_http` / `otlp_grpc`	`api.honeycomb.io:443` (EU: `api.eu1.honeycomb.io`)	`x-honeycomb-team: <key>`	OTLP	OTLP	Strong at high-cardinality trace queries

Rule of thumb: if a vendor accepts plain OTLP, use `otlp_http` with its endpoint and header. Use a vendor exporter only when the vendor needs extra processing (Datadog) or its OTLP intake lacks a signal you need.

Datadog Two ways from the collector

Datadog's APM pages rely on "trace stats" (hit counts, errors, latency per resource) computed from spans. The `datadog` connector computes them in the collector; the `datadog` exporter sends spans, stats and metrics. The alternative is to run the Datadog Agent and point the collector (or apps) at its OTLP port.



Way 1 — collector only

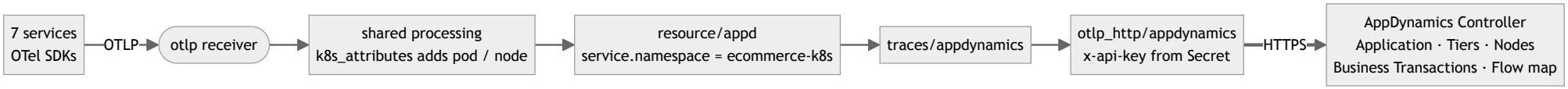
- No extra pods; everything in `otel-gateway.yaml`.
- Needs the contrib collector image (we already use it).
- Infra metrics need extra receivers (kubeletstats, k8s_cluster).

Way 2 — Datadog Agent

- Agent adds Datadog's own host, container and Kubernetes views.
- Collector keeps plain `otlp` → simple, vendor-neutral.
- Extra DaemonSet on every node (memory cost).

AppDynamics Mapping OpenTelemetry to AppD's model

AppDynamics organises everything as Application → Tier → Node and Business Transactions. Its OpenTelemetry ingest maps resource attributes onto that model, so the collector must set them before exporting. Spans become Business Transactions in the Controller.



OPENTELEMETRY ATTRIBUTE	APPDYNAMICS CONCEPT	IN THIS SHOP
`service.namespace`	Application	`ecommerce-k8s` (set by the resource/appd processor)
`service.name`	Tier	api-gateway, cart-service, order-service, …
`service.instance.id` / `k8s.pod.name`	Node	each pod, e.g. order-service-6b58…
server / entry spans	Business Transactions	POST /api/cart/{id}/checkout, GET /api/products, …

Steps

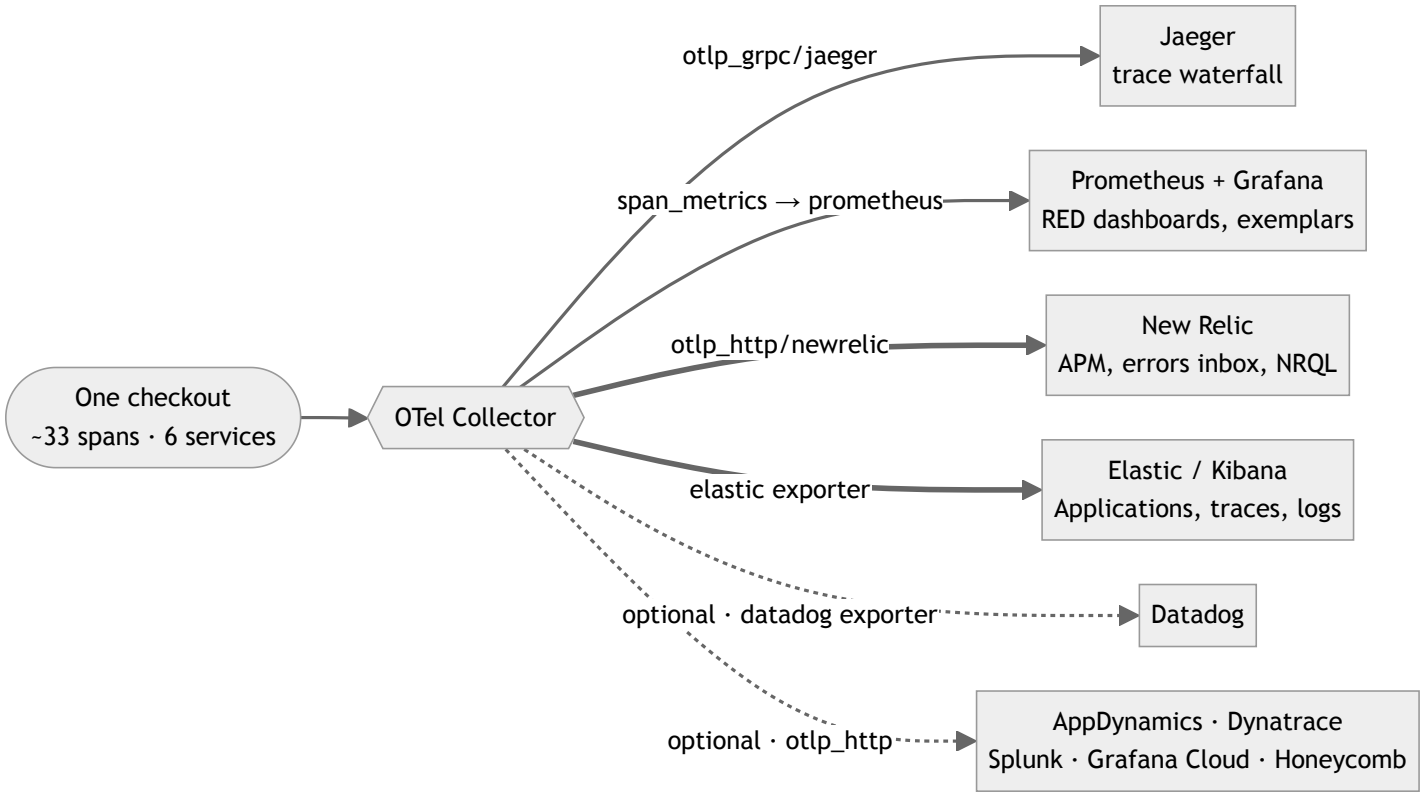
- In the Controller: enable OpenTelemetry, copy the OTel endpoint and API key.
- Secret `appd-otel-key`; exporter + `resource/appd` processor behind `appdynamics.enabled`.
- Verify: Tiers & Nodes, Business Transactions, flow map.

Good to know

- AppD's strongest features (code-level snapshots) come from its native agents; OTel gives traces and flow maps.
- Cisco also offers an AppDynamics distribution of the OTel collector.
- Same collector can still feed Jaeger, New Relic and Elastic.

End state: one collector, as many backends as you switch on

Each destination has its own pipeline and retries independently, so the same trace can be compared across tools and an outage in one backend never affects the others.



Why it stays simple

- Apps only know one address: the collector. New backends are collector settings.
- Every backend has an on/off switch in `charts/observability-config/values.yaml`.
- Keys live in Secrets; nothing secret is committed to Git.

Recommendation

- **Option B** to learn: free, no time limit, data stays local, and a third operator (ECK) next to the Prometheus and OpenTelemetry operators.
- **Option A** to see Elastic's UI quickly without extra memory.